T.C.

MİLLÎ SAVUNMA ÜNİVERSİTESİ

ALPARSLAN SAVUNMA BİLİMLERİ VE MİLLÎ GÜVENLİK ENSTİTÜSÜ

İHA Görüntülerinden Askeri Araç ve Personel Tespiti: Sızıntısız Çok Kaynaklı Veri Seti, YOLO26 ile Fine-Tuning ve TensorRT FP16 Quantization

Tolgahan KELEŞ

Öğrenci No: 26401212013

ARA SINAV PROJE RAPORU

Robotik ve Otonom Sistemler Mühendisliği Bölümü

İnsansız Hava Sistemleri Teknolojisi Dersi

Ders Sorumlusu

Dr. Öğr. Üyesi Süleyman TUNÇEL

15 Kasım 2026

İÇİNDEKİLER

SİMGE LİSTESİ ..... **iii**
KISALTMA LİSTESİ ..... **iv**
ŞEKİL LİSTESİ ..... **v**
TABLO LİSTESİ ..... **vi**
ÖZET ..... **vii**
ABSTRACT ..... **viii**
1 GİRİŞ **1**
2 İLGİLİ ÇALIŞMALAR **3**
2.1 Tek Aşamalı Nesne Tespiti 3
2.2 Havadan Nesne Tespiti 3
2.3 Quantization 3
2.4 Veri Sızıntısı 3
3 VERİ SETİ **4**
3.1 Kaynaklar ve Sınıf Eşlemesi 4
3.2 Veri Kalitesi Sorunları 5
3.3 Sızıntısız Birleştirme 6
4 YÖNTEM **9**
4.1 Model 9
4.2 Eğitim 9
4.3 TensorRT FP16 Quantization 10
4.4 Değerlendirme Ölçütleri 10
5 DENEYLER VE SONUÇLAR **12**
5.1 Deney Düzeni 12
5.2 Eğitim Dinamikleri 12
5.3 Test Başarımı 13
5.4 TensorRT FP16 Quantization 14
5.5 Nitel Sonuçlar ve Canlı Tespit 16
6 TARTIŞMA VE SINIRLILIKLAR **18**
6.1 Personel Tespiti 18
6.2 Model Ölçeği 18
6.3 Sınırlılıklar 18
7 SONUÇ **19**
KAYNAKÇA **20**

SİMGE LİSTESİ

B_p, B_g	Tahmin edilen ve gerçek sınırlayıcı kutu
C	Sınıf kümesi
d_H	Hamming uzaklığı
P	Kesinlik (precision)
R	Duyarlılık (recall)
s, e, m	FP16 sayısının işaret, üs ve kesir alanları
t	IoU eşiği

KISALTMA LİSTESİ

AP	Average Precision (Ortalama Kesinlik)
dHash	Difference Hash (Fark Hash'i)
FP16 / FP32	16 / 32 bit kayan noktalı sayı biçimi
FPS	Frames Per Second (Saniyedeki Kare Sayısı)
FPV	First Person View (Birinci Şahıs Görüşü)
GPU	Graphics Processing Unit (Grafik İşlem Birimi)
IFV	Infantry Fighting Vehicle (Piyade Muharebe Aracı)
INT8	8 bit tam sayı biçimi
IoU	Intersection over Union (Kesişim / Birleşim)
İHA	İnsansız Hava Aracı
mAP	mean Average Precision (Ortalama AP)
MD5	Message-Digest Algorithm 5
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
RTSP	Real Time Streaming Protocol
SAHI	Slicing Aided Hyper Inference
YOLO	You Only Look Once

ŞEKİL LİSTESİ

Şekil 1.1 Önerilen sistemin genel akışı. 2
Şekil 3.1 Birleştirilmiş veri setinden örnek görüntüler ve etiket kutuları. 5
Şekil 3.2 Veri setinin kaynak, bölme ve nesne boyutu dağılımları. 7
Şekil 5.1 YOLO26n ve YOLO26l modellerinin eğitim eğrileri. 13
Şekil 5.2 Test bölmesinde normalize karışıklık matrisleri. 14
Şekil 5.3 PyTorch FP32 ile TensorRT FP16 karşılaştırması. 15
Şekil 5.4 YOLO26l (TensorRT FP16) modelinin test görüntülerindeki tahminleri. 17

TABLO LİSTESİ

Tablo 3.1 Kullanılan veri setleri. 4
Tablo 3.2 Sızıntısız veri seti birleştirme algoritması. 6
Tablo 3.3 Bölme istatistikleri. 7
Tablo 4.1 Eğitim hiperparametreleri. 10
Tablo 5.1 Test bölmesinde başarım (PyTorch FP32). 13
Tablo 5.2 Sınıf bazında AP (PyTorch FP32 ve TensorRT FP16). 14
Tablo 5.3 PyTorch ve TensorRT biçimlerinin karşılaştırması (test bölmesi). 15

ÖZET

İHA Görüntülerinden Askeri Araç ve Personel Tespiti: Sızıntısız Çok Kaynaklı Veri Seti, YOLO26 ile Fine-Tuning ve TensorRT FP16 Quantization

Tolgahan KELEŞ

Robotik ve Otonom Sistemler Mühendisliği Bölümü

İnsansız Hava Sistemleri Teknolojisi Dersi

Ara Sınav Proje Raporu

Ders Sorumlusu: Dr. Öğr. Üyesi Süleyman TUNÇEL

Bu çalışmada insansız hava aracı (İHA) görüntülerinde askeri araç ve personel tespiti yapan, gömülü GPU'larda gerçek zamanlı çalışabilecek bir sistem geliştirilmiştir. Beş açık veri seti birleştirilerek 36 997 görüntülük, 2 sınıflı bir veri seti oluşturulmuştur. Bölmeler arası sızıntıyı önlemek için MD5 ve algısal hash (dHash) ile 5 809 tekrar görüntü ayıklanmış, yakın kopyalar ve aynı videodan gelen kareler birleşim-bul yapısıyla gruplanarak kaynak bazında katmanlı olarak bölünmüştür.

COCO ön eğitimli YOLO26n ve YOLO26l modelleri Ultralytics Platform üzerinde fine-tuning ile eğitilmiştir. En iyi model (YOLO26l) test setinde 0.821 mAP@0.5 ve 0.566 mAP@0.5:0.95 elde etmiştir. Modeller TensorRT FP16 formatına dönüştürüldüğünde model çıkarım süresi YOLO26n'de 4.0, YOLO26l'de 4.4 kat kısalmış, uçtan uca hız sırasıyla 123 ve 79 FPS'e ulaşmıştır. mAP@0.5:0.95'teki değişim ise her iki modelde de 0.001'in altında kalmıştır. PyTorch FP16 ile yapılan karşılaştırma, hızlanmanın büyük kısmının yarı hassasiyetin kendisinden değil TensorRT'nin grafik optimizasyonlarından geldiğini göstermiştir.

Anahtar Kelimeler: İnsansız hava aracı, nesne tespiti, YOLO26, veri sızıntısı, algısal hash, TensorRT, FP16 quantization.

ABSTRACT

Military Vehicle and Personnel Detection from UAV Imagery: A Leakage-Free Multi-Source Dataset, YOLO26 Fine-Tuning and TensorRT FP16 Quantization

Tolgahan KELEŞ

Department of Robotics and Autonomous Systems Engineering

Unmanned Aerial Systems Technology Course

Midterm Project Report

Lecturer: Asst. Prof. Dr. Süleyman TUNÇEL

This study presents a system that detects military vehicles and personnel in unmanned aerial vehicle (UAV) imagery and runs in real time on embedded GPUs. Five public datasets were merged into a two-class dataset of 36,997 images. To prevent leakage between splits, 5,809 duplicate images were removed using MD5 and perceptual hashing (dHash), and near-duplicates and frames from the same video were grouped with a union-find structure before a source-stratified split.

COCO-pretrained YOLO26n and YOLO26l models were fine-tuned on Ultralytics Platform. The best model (YOLO26l) reached 0.821 mAP@0.5 and 0.566 mAP@0.5:0.95 on the test set. Converting the models to TensorRT FP16 reduced model inference time by a factor of 4.0 for YOLO26n and 4.4 for YOLO26l, reaching end-to-end speeds of 123 and 79 FPS, while the change in mAP@0.5:0.95 stayed below 0.001 for both models. A comparison with PyTorch FP16 showed that most of the speed-up comes from TensorRT's graph optimizations rather than from half precision itself.

Keywords: Unmanned aerial vehicle, object detection, YOLO26, data leakage, perceptual hashing, TensorRT, FP16 quantization.

1

GİRİŞ

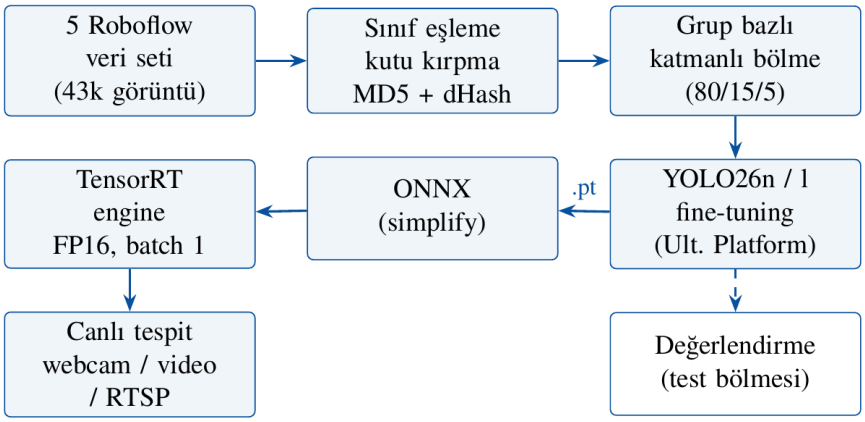
İnsansız hava araçları (İHA) keşif, gözetleme ve hedef tespiti görevlerinde yaygın olarak kullanılmaktadır. Bu görevlerin otonom yürütülebilmesi için, kameradan gelen görüntülerde ilgili nesnelerin uçuş bilgisayarı üzerinde ve gerçek zamanlı olarak tespit edilmesi gerekir. Havadan çekilen görüntülerde nesneler küçük, bakış açıları değişken ve arka planlar karmaşıktır [4], [5]. Ayrıca İHA üzerindeki hesaplama kaynakları sınırlı olduğundan, modelin doğruluğu kadar gecikmesi ve bellek ihtiyacı da belirleyicidir.

Bu çalışmada açık kaynak veri setleri kullanılarak askeri araç ve personel tespiti yapan bir sistem geliştirilmiş ve bir GPU üzerinde gerçek zamanlı çalışacak şekilde optimize edilmiştir. Açık veri setleri hızlı başlangıç sağlasa da hazır bölmeleri (train/valid/test) çoğu zaman güvenilir değildir: aynı görüntünün augment edilmiş kopyaları veya aynı videodan ardışık kareler farklı bölmelere düşebilmekte ve test başarımı olduğundan yüksek ölçülmektedir. Bu nedenle çalışmanın önemli bir kısmı veri hazırlığına ayrılmıştır.

Çalışmanın katkıları şunlardır:

- Beş farklı Roboflow veri seti, sınıf eşlemesi yapılarak 2 sınıflı (*military-vehicle*, *person*) tek bir veri setinde birleştirilmiştir. MD5 ve algısal hash (dHash) ile kaynaklar arası tekrarlar ayıklanmış, benzer görüntüler ve video kareleri aynı bölmeye zorlanarak sızıntısız bir bölme elde edilmiştir.
- COCO ön eğitimli YOLO26n ve YOLO26l modelleri Ultralytics Platform üzerinde fine-tuning ile eğitilmiş, eğitim dinamikleri ve test başarımları karşılaştırılmıştır.
- Modeller TensorRT FP16 formatına dönüştürülmüş; doğruluk, gecikme ve model boyutu, hassasiyetin ve derlemenin etkilerini ayırmak için PyTorch FP32 ve PyTorch FP16 ile birlikte ölçülmüştür.
- Kamera, video ve RTSP akışı üzerinde çalışan bir canlı tespit uygulaması geliştirilmiştir.

Önerilen sistemin veri hazırlamadan dağıtıma kadar genel akışı Şekil 1.1'de özetlenmiştir.



Şekil 1.1 Önerilen sistemin genel akışı.

2

İLGİLİ ÇALIŞMALAR

2.1 Tek Aşamalı Nesne Tespiti

YOLO ailesi [1] tespiti tek bir regresyon problemi olarak ele alarak gerçek zamanlı çalışmayı mümkün kılmıştır. Ultralytics YOLO sürümleri [2] bu yaklaşımı eğitim, doğrulama ve dışa aktarma araçlarıyla birlikte sunar. Bu çalışmada kullanılan YOLO26, NMS gerektirmeyen uçtan uca bir tahmin başlığı ve kenar cihazlara yönelik sadeleştirilmiş bir mimari sunar [3].

2.2 Havadan Nesne Tespiti

VisDrone [4] ve DOTA [5] gibi veri setleri, havadan görüntülerde küçük nesnelerin ve yoğun sahnelerin tespiti zorlaştırdığını göstermiştir. Küçük nesneler için görüntüyü parçalara ayırarak çıkarım yapan SAHI [6] gibi yöntemler önerilmiştir.

2.3 Quantization

Ağırlık ve aktivasyonların daha düşük bitle gösterilmesi bellek ve hesap maliyetini azaltır [7], [8]. FP16 (yarı hassasiyet) kalibrasyon gerektirmez ve çoğu ağda doğruluğu korur [9]. INT8 ise daha fazla hızlanma sağlar ancak kalibrasyon verisi gerektirir. NVIDIA TensorRT [10], katman birleştirme ve çekirdek seçimi gibi grafik optimizasyonlarını düşük hassasiyetle birleştirir.

2.4 Veri Sızıntısı

Eğitim ve test kümelerinde neredeyse aynı görüntülerin bulunması başarımı olduğundan yüksek gösterir; CIFAR test setinde bu etki ayrıntılı olarak incelenmiştir [11]. Algısal hash (ör. dHash [12]) bu tür yakın kopyaları ucuz bir şekilde bulmak için yaygın olarak kullanılır.

3

VERİ SETİ

3.1 Kaynaklar ve Sınıf Eşlemesi

Çalışmada beş Roboflow Universe [13] veri seti kullanılmıştır (Tablo 3.1). Kaynakların sınıf adları birbirinden farklı olduğundan tüm askeri araç türleri (tank, zırhlı araç, IFV, askeri kamyon, topçu) *military-vehicle*, asker ve personel ise *person* sınıfına eşlenmiştir. Görevle ilgisiz sınıflar (sivil araç, uçak, helikopter, FPV drone, sivil kişi) kutularıyla birlikte atılmıştır.

Tablo 3.1 Kullanılan veri setleri.

Kısa ad	Roboflow projesi (sürüm)	Görüntü sayısı[a]
huyen	huyen-dinh/military-vehicle-rstvv (2)	9 434
gzhet	test-fo7by/military-gzhet (1)	12 786
roman	romanpertdrone/military-lyxbx (2)	6 561
view6	datasets-tqcu8/military-view6 (2)	3 827
mamad	mamad-vira/military-vehicle-m5bvw (1)	4 389
Toplam		36 997

[a] Birleştirme ve tekrar ayıklama sonrası kalan görüntü sayısı.

Şekil 3.1 her kaynaktan örnek görüntüleri ve etiket kutularını göstermektedir. Veri seti yerden çekilmiş fotoğraflar, gri tonlu ve termal benzeri drone görüntüleri, FPV kamera kareleri ve sentetik görünümlü sahneler içermektedir.



Şekil 3.1 Birleştirilmiş veri setinden örnek görüntüler ve etiket kutuları.

Şekil 3.1'de turuncu kutular *military-vehicle*, sarı kutular *person* sınıfını göstermektedir. Görüntüler sırasıyla (a) yerden çekilmiş fotoğraf, (b) gri tonlu havadan görüntü, (c) FPV drone kamerası, (d) uzak mesafeden çekilmiş sahne, (e) sentetik görünümlü havadan sahne ve (f) personel örnekleridir. (f)'de soldan ikinci asker etiketlenmemiştir; bu durum açık veri setlerindeki etiket gürültüsüne bir örnektir.

3.2 Veri Kalitesi Sorunları

Kaynak veri setleri incelendiğinde birleştirmeden önce çözülmesi gereken şu sorunlar bulunmuştur:

- Sınıf Adı Hataları ve Az Örnekli Sınıflar

huyen veri setinde *amoured-vehicle* adlı yazım hatalı bir sınıf ve yalnızca 49 kutu içeren *artillery* sınıfı bulunmaktadır. Her ikisi de *military-vehicle* sınıfına eşlenmiştir.

- Augment Kopyaları

Roboflow, augmentation ile ürettiği kopyalara kaynak görüntünün adını taşıyan dosya adları verir (*<ad>_jpg.rf.<hash>.jpg*). Hazır bölmelerde aynı kaynaktan türetilmiş kopyaların farklı bölmelere dağıldığı görülmüştür.

- Hatalı Dosya ve Etiketler

Bozuk görüntü dosyaları, birebir tekrar eden görüntüler ve görüntü sınırlarının dışına taşan kutular bulunmaktadır. Bozuk dosyalar atılmış, kutular [0, 1] aralığına kırpılmıştır.

- Kaynaklar Arası Tekrar

Aynı görüntü birden fazla kaynakta, bazen farklı boyutlandırma veya sıkıştırmayla yer almaktadır.

3.3 Sızıntısız Birleştirme

Birleştirme sırasında üç tür sızıntı kaynağı ele alınmıştır: farklı kaynaklarda aynı görüntünün bulunması, Roboflow augment kopyaları ve aynı videodan alınmış ardışık kareler. Uygulanan yöntemin adımları Tablo 3.2'de özetlenmiştir.

Her görüntü için 64 bitlik dHash [12] hesaplanır: görüntü gri tona çevrilip 9×8 boyutuna küçültülür ve her satırdaki komşu piksellerin karşılaştırılmasıyla 64 bit elde edilir. İki görüntü arasındaki benzerlik, hash'ler arasındaki Hamming uzaklığı d_H ile ölçülür. Yaklaşık 43 bin görüntü için tüm çiftleri karşılaştırmak (yaklaşık 9×10^8 çift) pahalı olduğundan hash 8 adet 8 bitlik parçaya bölünür ve yalnızca en az bir parçası aynı olan görüntüler karşılaştırılır. Güvercin yuvası ilkesine göre $d_H \leq 7$ olan iki hash'in 8 parçadan en az birinin aynı olması gerektiğinden, bu yöntem $d_H \leq 6$ eşiği için hiçbir aday çifti kaçırmaz.

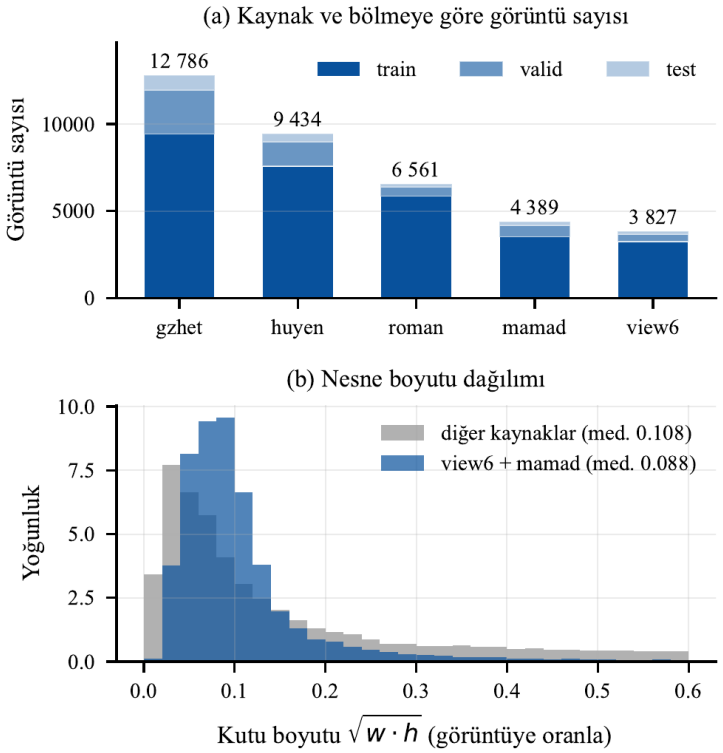
Tablo 3.2 Sızıntısız veri seti birleştirme algoritması.

Adım	İşlem
1	Etiketleri eşle, ilgisiz sınıfları at, kutuları [0, 1] aralığına kırp.
2	Her görüntü i için $MD5_i$ ve 64 bit $dHash_i$ hesapla.
3	Birleşim-bul (union-find) yapısı U oluştur.
4	Aynı 8 bitlik hash parçasını paylaşan her (a, b) çifti için d = popcount($dHash_a$ XOR $dHash_b$) hesapla.
5	$MD5_a = MD5_b$ veya $d \leq 6$ ise a ve b'yi U'da birleştir.
6	$MD5_a = MD5_b$ veya $d \leq 2$ ise öncelikli olmayanı *tekrar* olarak işaretle.
7	Aynı Roboflow kaynak adını veya aynı video adını taşıyanları U'da birleştir.
8	Tekrarları sil; U'nun bileşenlerini *grup* kabul et.
9	Grupları baskın kaynaklarına göre katmanla, 80/15/5 oranında böl (tohum 42).

$d_H \leq 2$ olan çiftler tekrar kabul edilip tek kopyası tutulmuş, bu şekilde 5 809 görüntü atılmıştır. $2 < d_H \leq 6$ olan çiftler ise silinmemiş ancak aynı gruba alınmıştır. Aynı videodan alınmış kareler dosya adındaki desenlerden (ör. *klip_mp4-15*) tanınarak aynı gruba konulmuştur. Son olarak gruplar kaynak bazında katmanlı olarak bölünmüş, böylece her kaynak her bölmede kendi payı oranında temsil edilmiştir. Sonuç veri setinin istatistikleri Tablo 3.3 ve Şekil 3.2'de verilmiştir.

Tablo 3.3 Bölme istatistikleri.

Bölme	Görüntü	military-vehicle kutu	person kutu	Toplam kutu
train	29 603	43 062	24 858	67 920
valid	5 547	8 784	3 671	12 455
test	1 847	3 338	1 226	4 564
Toplam	36 997	55 184	29 755	84 939



Şekil 3.2 Veri setinin kaynak, bölme ve nesne boyutu dağılımları.

Şekil 3.2(a) kaynak ve bölmeye göre görüntü sayılarını göstermektedir; katmanlı bölme sayesinde her kaynak her bölmede yaklaşık 80/15/5 oranındadır. Şekil 3.2(b) ise kutu

boyutunun (kutu genişliği ile yüksekliğinin geometrik ortalamasının görüntü boyutuna oranı) dağılımını vermektedir. view6 ve mamad kaynaklarında nesneler belirgin şekilde daha küçüktür ve dağılım dardır; diğer kaynaklar hem çok küçük hem de kadrajı kaplayan nesneler içerir.

4

YÖNTEM

4.1 Model

YOLO26 [3], Ultralytics'in kenar cihazlara yönelik son nesil tespit modelidir. NMS adımını gerektirmeyen uçtan uca bir tahmin başlığına sahiptir; bu sayede son işleme süresi kısalır ve modelin TensorRT gibi formatlara aktarımı kolaylaşır. Eğitimde kutu regresyonu için IoU tabanlı kayıp ve L1 kaybı, sınıflandırma için ikili çapraz entropi kullanılır. Çalışmada iki ölçek kullanılmıştır: hız odaklı YOLO26n ve doğruluk odaklı YOLO26l. Her iki model de COCO [14] üzerinde ön eğitimli ağırlıklarla başlatılmıştır.

4.2 Eğitim

Eğitimler Ultralytics Platform'un bulut GPU'larında yapılmıştır. Hiperparametreler Tablo 4.1'de özetlenmiştir. Her iki modelde de 640 piksel giriş boyutu, 100 epoch, mozaik augmentation (son 10 epoch'ta kapalı), yatay çevirme, ölçekleme, HSV renk oynatma ve rastgele silme (*erasing*) kullanılmıştır. Optimizasyon algoritması olarak YOLO26 ile birlikte gelen MuSGD kullanılmıştır. Batch boyutu model büyüklüğüne göre farklılık göstermektedir. Erken durdurma kullanılmamış, doğrulama mAP@0.5:0.95 değeri en yüksek olan epoch'un ağırlıkları saklanmıştır. Tüm eğitim geçmişi (epoch başına kayıplar, ölçütler ve öğrenme oranları) model dosyalarının içinde saklandığından, eğitim eğrileri doğrudan bu dosyalardan elde edilmiştir.

Tablo 4.1 Eğitim hiperparametreleri.

Parametre	YOLO26n	YOLO26l
Optimizasyon	MuSGD	MuSGD
lr_0 / lr_f	0.01 / 0.01	0.01 / 0.01
Batch	275	82
Epoch (en iyi)	100 (83)	100 (62)
Eğitim süresi	58.3 dk	3.38 sa
Model boyutu	5.5 MB	53.0 MB

Ortak ayarlar: imgsz 640, momentum 0.937, ağırlık çürümesi 5×10^{-4}, warmup 3 epoch, mozaik 1.0 (son 10 epoch kapalı), fliplr 0.5, scale 0.5, translate 0.1, erasing 0.4, AMP.

4.3 TensorRT FP16 Quantization

IEEE 754 yarı hassasiyetli sayı biçimi (FP16) 1 işaret, 5 üs ve 10 kesir bitinden oluşur:

$$x = (-1)^s \cdot 2^{e-15} \cdot \left(1 + \frac{m}{2^{10}}\right) \tag{4.1}$$

Bu biçimde temsil edilebilen en büyük değer 65 504, 1 civarındaki göreli hassasiyet ise $2^{-10} \approx 9.8\times10^{-4}$'tür. FP32'ye göre bellek yarıya iner ve GPU'ların tensor çekirdekleri kullanılabilir. INT8'den farklı olarak aktivasyon aralıklarının kalibrasyon verisiyle belirlenmesine gerek yoktur.

Dönüşüm iki adımda yapılmıştır: (i) PyTorch modeli ONNX grafiğine aktarılıp sadeleştirilmiş (*onnx-simplifier*), (ii) TensorRT ile batch 1 ve 640×640 sabit giriş boyutuna sahip, FP16 çekirdeklere izin verilmiş bir engine derlenmiştir. Test GPU'sunun belleği 6 GB olduğundan derleme çalışma alanı 2 GB ile sınırlandırılmıştır. Hassasiyetin ve TensorRT grafik optimizasyonlarının etkilerini ayırmak için her model üç biçimde ölçülmüştür: PyTorch FP32, PyTorch FP16 (*half=True*) ve TensorRT FP16.

4.4 Değerlendirme Ölçütleri

Bir tahmin, aynı sınıftan bir gerçek kutuyla kesişim/birleşim oranı (IoU) eşikten büyükse doğru kabul edilir:

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|} \tag{4.2}$$

Kesinlik P = TP / (TP + FP) ve duyarlılık R = TP / (TP + FN) olarak tanımlanır. Bir sınıfın ortalama kesinliği (AP), kesinlik-duyarlılık eğrisinin altında kalan alandır ve 101 noktalı enterpolasyonla hesaplanır [14]. mAP@0.5 tüm sınıfların IoU = 0.5'teki AP ortalamasıdır; mAP@0.5:0.95 ise 0.50'den 0.95'e 0.05 adımla 10 IoU eşiğindeki ortalamadır:

$$\mathrm{mAP}_{0.5:0.95} = \frac{1}{|C|}\sum_{c \in C}\frac{1}{10}\sum_{t \in \{0.50,\ldots,0.95\}} \mathrm{AP}_c(t) \tag{4.3}$$

Hız, GPU'da başka bir iş yükü yokken, test bölmesinden 500 görüntü üzerinde batch 1 ile ölçülmüştür. İlk 50 görüntü ısınma için ayrıca çalıştırılıp ölçüme katılmamıştır. Görüntü başına ortalama model çıkarım süresi (ms) ve görüntünün diskten okunması, ön işleme ve son işleme dahil uçtan uca süre raporlanmıştır.

5

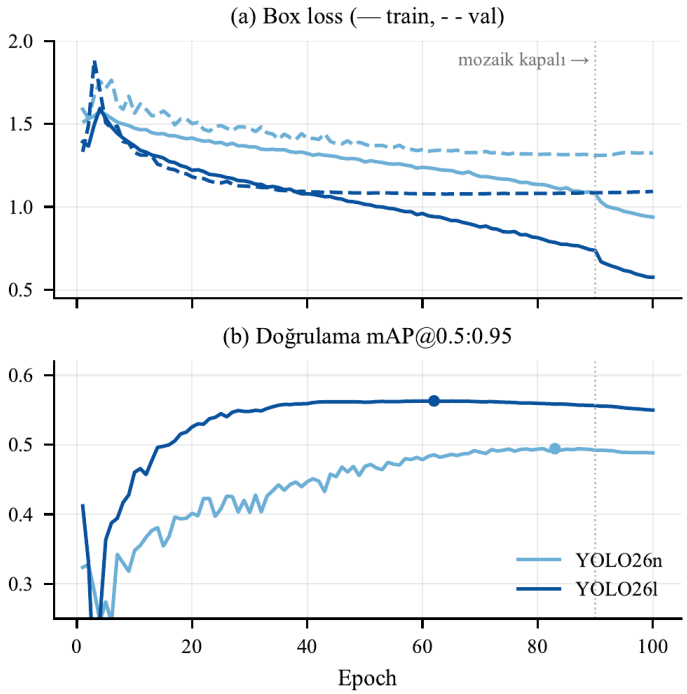
DENEYLER VE SONUÇLAR

5.1 Deney Düzeni

Tüm değerlendirmeler, eğitim ve model seçimi sırasında hiç kullanılmamış olan *test* bölmesinde (1 847 görüntü, 4 564 kutu) yapılmıştır. Ölçümler giriş boyutu 640 ve batch 1 ile, 6 GB belleğe sahip bir NVIDIA GeForce RTX 3050 Laptop GPU üzerinde, Ultralytics 8.4.203 ve TensorRT ile yapılmıştır.

5.2 Eğitim Dinamikleri

Şekil 5.1 iki modelin eğitim eğrilerini göstermektedir. Her iki model de ilk 20 epoch'ta hızla yakınsamış, ardından doğrulama mAP'i yavaş artmıştır. Son 10 epoch'ta mozaik augmentation'ın kapatılmasıyla eğitim kaybı belirgin bir şekilde düşmüştür. Doğrulama kaybında ise bu düşüş görülmemektedir. YOLO26l'de doğrulama box kaybı 64. epoch'ta en düşük değerine (1.077) ulaşmış, sonra 1.093'e yükselmiştir. mAP@0.5:0.95 de 62. epoch'tan sonra 0.563'ten 0.550'ye gerilemiştir. Bu hafif aşırı öğrenme, en iyi epoch'un ağırlıklarının saklanmasıyla telafi edilmiştir. YOLO26n ise 83. epoch'ta en iyi değerine ulaşmış ve eğitim sonuna kadar aşırı öğrenme göstermemiştir. Aynı veriyle eğitildiği halde YOLO26l'nin doğrulama kaybı tüm eğitim boyunca belirgin şekilde daha düşüktür. Bu da büyük modelin kapasitesinin bu veri setinde fark yarattığını göstermektedir.



Şekil 5.1 YOLO26n ve YOLO26l modellerinin eğitim eğrileri.

Şekil 5.1(a)'da eğitim (düz çizgi) ve doğrulama (kesikli çizgi) box kaybı, Şekil 5.1(b)'de doğrulama mAP@0.5:0.95 değeri verilmiştir; nokta, saklanan en iyi epoch'u gösterir. Noktalı dikey çizgi mozaik augmentation'ın kapatıldığı 90. epoch'u göstermektedir.

5.3 Test Başarımı

Tablo 5.1, PyTorch FP32 modellerinin test sonuçlarını özetlemektedir. YOLO26l, YOLO26n'e göre mAP@0.5:0.95'i 0.497'den 0.566'ya (+0.069), mAP@0.5'i 0.773'ten 0.821'e çıkarmıştır. Kesinlik ve duyarlılık da sırasıyla 0.815'ten 0.850'ye ve 0.721'den 0.778'e yükselmiştir.

Tablo 5.1 Test bölmesinde başarım (PyTorch FP32).

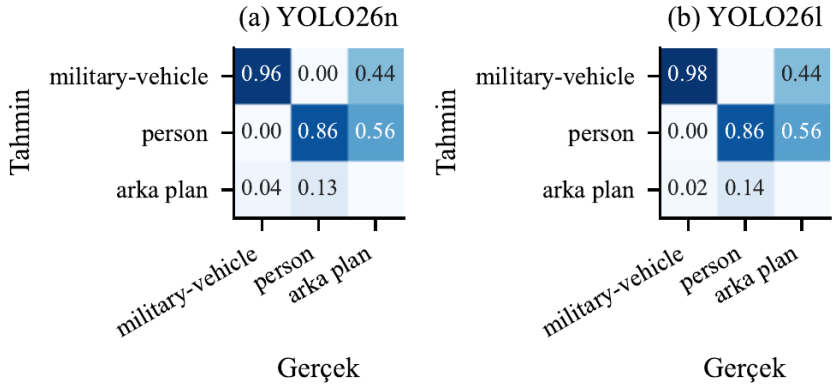
Model	*P*	*R*	mAP@0.5	mAP@0.5:0.95
YOLO26n	0.815	0.721	0.773	0.497
YOLO26l	0.850	0.778	0.821	0.566

Sınıf bazında sonuçlar Tablo 5.2'de verilmiştir. Her iki modelde de daha zor sınıf *person*'dır: YOLO26l'de araç sınıfının AP@0.5'i 0.945 iken personelinki 0.697'dir. Karışıklık matrisleri (Şekil 5.2) iki sınıfın birbiriyle neredeyse hiç karışmadığını, hataların arka planla ilgili olduğunu göstermektedir. Gerçek personelin YOLO26n'de

%13'ü, YOLO26l'de %14'ü hiç tespit edilmemiştir. Araçlarda bu oran %4'ten %2'ye inmektedir. Her iki modelde de yanlış pozitiflerin %56'sı *person* olarak işaretlenmiştir.

Tablo 5.2 Sınıf bazında AP (PyTorch FP32 ve TensorRT FP16).

Model	Sınıf	AP@0.5	AP@0.5:0.95 FP32	AP@0.5:0.95 FP16
YOLO26n	military-vehicle	0.901	0.631	0.629
YOLO26n	person	0.646	0.362	0.364
YOLO26l	military-vehicle	0.945	0.716	0.714
YOLO26l	person	0.697	0.416	0.420



Şekil 5.2 Test bölmesinde normalize karışıklık matrisleri.

Şekil 5.2'de sütunlar gerçek sınıfa göre normalize edilmiştir. *Arka plan* sütunu yanlış pozitiflerin dağılımını, *arka plan* satırı ise kaçırılan nesneleri gösterir.

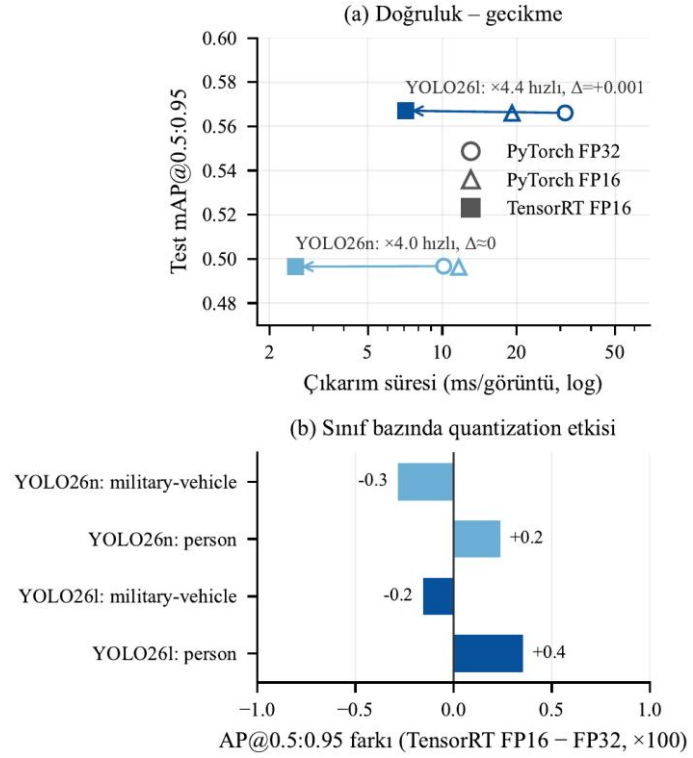
5.4 TensorRT FP16 Quantization

Tablo 5.3 ve Şekil 5.3 quantization sonuçlarını göstermektedir. TensorRT FP16, model çıkarım süresini YOLO26n'de 10.1 ms'den 2.6 ms'ye (4.0 kat), YOLO26l'de 31.4 ms'den 7.1 ms'ye (4.4 kat) indirmiştir. Uçtan uca hız YOLO26n'de 64'ten 123 FPS'e, YOLO26l'de 27'den 79 FPS'e çıkmıştır. Doğruluk ise neredeyse hiç değişmemiştir. mAP@0.5:0.95 farkı YOLO26n'de −0.0002, YOLO26l'de +0.001'dir ve sınıf bazındaki farklar ±0.4 puanı geçmemektedir. FP16'nın 10 bitlik kesir hassasiyeti bu modellerin ağırlık ve aktivasyonları için yeterlidir.

Tablo 5.3 PyTorch ve TensorRT biçimlerinin karşılaştırması (test bölmesi).

Model	Biçim	Boyut (MB)	Çıkarım (ms)[a]	Hızlanma (kat)	Uçtan uca FPS	mAP@0.5	mAP@0.5:0.95
YOLO26n	PyTorch FP32	5.5	10.1	1.0	64	0.773	0.497
YOLO26n	PyTorch FP16	5.5[b]	11.7	0.9	58	0.772	0.496
YOLO26n	TensorRT FP16	62.0	2.6	4.0	123	0.772	0.496
YOLO26l	PyTorch FP32	53.0	31.4	1.0	27	0.821	0.566
YOLO26l	PyTorch FP16	53.0[b]	19.2	1.6	41	0.822	0.566
YOLO26l	TensorRT FP16	201.6	7.1	4.4	79	0.818	0.567

[a] Model çıkarımı (ön/son işleme hariç), batch 1, boş GPU'da 500 görüntü ortalaması. Uçtan uca FPS görüntü okuma, ön ve son işlemeyi de içerir. [b] Aynı dosya; ağırlıklar yükleme sırasında FP16'ya çevrilir.



Şekil 5.3 PyTorch FP32 ile TensorRT FP16 karşılaştırması.

Şekil 5.3(a) test bölmesindeki doğruluğu ve çıkarım süresini (logaritmik ölçek) göstermektedir; oklar PyTorch FP32'den TensorRT FP16'ya geçişi belirtir. Şekil 5.3(b) sınıf bazında AP@0.5:0.95 değişimini (TensorRT FP16 − PyTorch FP32, 100 ile çarpılmış) vermektedir.

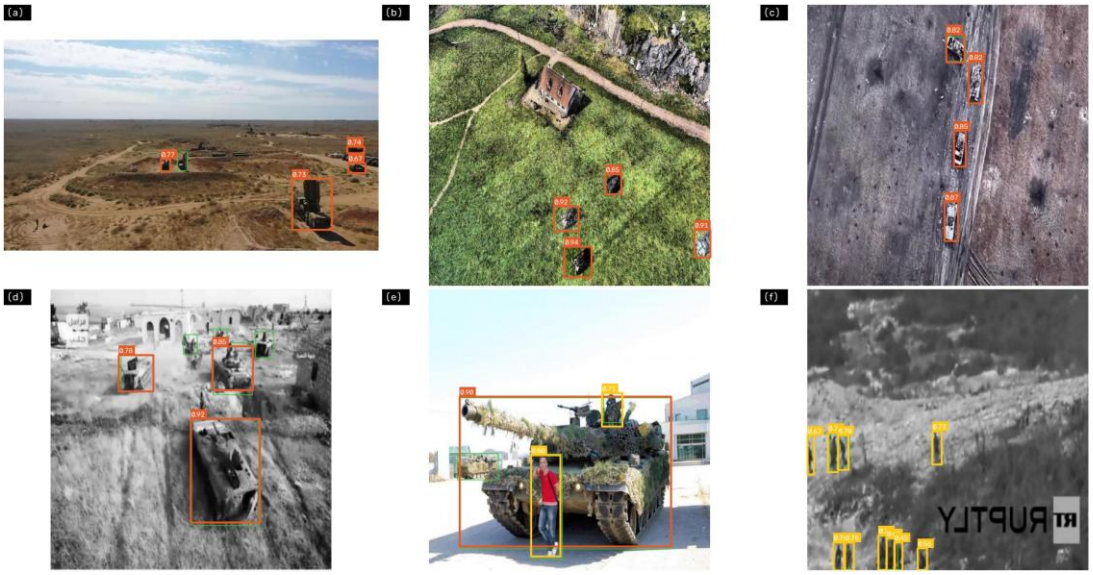
Hızlanmanın kaynağını ayırmak için modeller PyTorch'ta FP16 hassasiyetle (*half=True*) de ölçülmüştür. PyTorch FP16'nın doğruluğu FP32 ile aynıdır. Hız etkisi ise model büyüklüğüne bağlıdır: YOLO26l'de çıkarım süresi 31.4 ms'den 19.2 ms'ye inmiş (1.6 kat), küçük YOLO26n'de ise 10.1 ms'den 11.7 ms'ye çıkmıştır. Batch 1 ile çalışan küçük ağlarda süre, aritmetikten çok çekirdek çağrılarının ek yükü tarafından belirlenir; FP16 aritmetiği bu yükü azaltmaz. TensorRT ise bu yükü katman birleştirme ve donanıma özel çekirdek seçimi ile azaltmaktadır. Aynı FP16 hassasiyetinde TensorRT, PyTorch FP16'ya göre YOLO26l'de 2.7, YOLO26n'de 4.6 kat daha hızlıdır. Dolayısıyla hızlanmanın büyük bölümü yarı hassasiyetin kendisinden değil, TensorRT'nin grafik optimizasyonlarından gelmektedir.

Dosya boyutları beklenenden farklıdır: TensorRT engine dosyaları (62 ve 202 MB) PyTorch dosyalarından büyüktür. Bunun nedeni engine dosyasının yalnızca ağırlıkları değil, hedef GPU için derlenmiş çekirdekleri de içermesidir. Ağırlıkların kendisi FP16'da gerçekten yarıya inmektedir: ONNX dosyası YOLO26l için 99.4 MB'tan (FP32) 49.9 MB'a (FP16), YOLO26n için 9.9 MB'tan 5.0 MB'a düşmüştür.

Çıkarım süresi kısaldıkça toplam gecikmede diğer adımların payı artmaktadır. Görüntü başına ön işleme yaklaşık 1.2 ms, son işleme 1.5–1.8 ms, görüntünün diskten okunup çözülmesi ise yaklaşık 2.5 ms sürmektedir. Bu nedenle uçtan uca hızlanma, model çıkarımındaki hızlanmadan daha düşüktür: YOLO26n'de 1.9, YOLO26l'de 2.9 kat. TensorRT ile YOLO26n'in çıkarımı 2.6 ms'ye indiğinde, model dışındaki adımlar uçtan uca sürenin yaklaşık üçte ikisini oluşturmaktadır. Daha fazla hızlanma için bu adımların da GPU'ya taşınması veya kamera akışıyla paralel yürütülmesi gerekir.

5.5 Nitel Sonuçlar ve Canlı Tespit

Şekil 5.4, YOLO26l TensorRT FP16 modelinin test görüntülerindeki tahminlerini göstermektedir. Model havadan ve drone görüntülerindeki araçları (a–d) yüksek güvenle bulmaktadır. Hatalar özellikle uzak ve küçük nesnelerde (a, d) ve kalabalık personel gruplarında (f) yoğunlaşmaktadır.



Şekil 5.4 YOLO26l (TensorRT FP16) modelinin test görüntülerindeki tahminleri.

Şekil 5.4'te güven eşiği 0.35'tir. Kalın kutular tahminleri ve güven skorlarını (turuncu: *military-vehicle*, sarı: *person*), ince yeşil kutular gerçek etiketleri gösterir. (a) uzak mesafeden havadan sahnedir ve soldaki küçük araç kaçırılmıştır. (b) sentetik havadan sahne, (c) drone görüntüsüdür. (d) gri tonlu video karesidir ve arka plandaki küçük araçlar kaçırılmıştır. (e) yer görüntüsüdür ve arkadaki kısmen görünen araç kaçırılmıştır. (f) termal benzeri görüntüde personel grubunu göstermektedir.

Canlı tespit uygulaması webcam, video dosyası ve RTSP akışını (İHA video bağlantısı) kaynak olarak kabul etmektedir. Ekranda toplam FPS, kamera okuma süresi, model süresi ve sınıf başına tespit sayısı gösterilmektedir. Windows'ta DirectShow'un 720p çözünürlükte sıkıştırılmamış YUY2 biçimini seçmesi kamera FPS'ini yaklaşık 10'a düşürdüğünden MJPG biçimi zorlanmıştır. Bu deneyim, gerçek sistemde darboğazın modelden çok kamera ve G/Ç tarafında olabileceğini göstermektedir.

6

TARTIŞMA VE SINIRLILIKLAR

6.1 Personel Tespiti

Her iki modelde de *person* sınıfı daha düşük başarım göstermektedir (AP@0.5:0.95 0.36–0.42). Havadan bakıldığında insanlar birkaç pikselden oluşmakta, sıklıkla kamuflajlı ve kısmen örtülü olmaktadır. Ayrıca açık veri setlerinde personel etiketleri tutarsızdır (Şekil 3.1f). Etiket gürültüsü hem eğitimi hem de ölçülen başarımı olumsuz etkilemektedir.

6.2 Model Ölçeği

YOLO26l, YOLO26n'e göre test mAP@0.5:0.95'te 0.069 puan daha iyidir. Buna karşılık model yaklaşık 10 kat büyük, TensorRT'de ise yaklaşık 2.8 kat yavaştır (7.1 ms ve 2.6 ms). Yine de 79 FPS'lik uçtan uca hızıyla gerçek zamanlı çalışma sınırının oldukça üzerindedir. Gerçek zamanlı bir İHA uygulamasında hangi modelin seçileceği, hedef donanımın hesaplama gücüne ve küçük nesnelerin önemine bağlıdır.

6.3 Sınırlılıklar

- Hız ölçümleri bir laptop GPU'sunda yapılmıştır. Jetson gibi gömülü kartlarda engine'in o donanım için yeniden derlenmesi ve hızın orada ölçülmesi gerekir.
- INT8 quantization ve küçük nesneler için parçalı çıkarım denenmemiştir.

7

SONUÇ

Bu çalışmada İHA görüntülerinden askeri araç ve personel tespiti için veri hazırlamadan dağıtıma kadar uçtan uca bir sistem geliştirilmiştir. Beş açık veri seti algısal hash ve grup bazlı katmanlı bölme ile sızıntısız biçimde birleştirilmiş, YOLO26 modelleri bu veriyle fine-tuning ile eğitilmiştir. En iyi model test setinde 0.821 mAP@0.5 ve 0.566 mAP@0.5:0.95 elde etmiştir.

TensorRT FP16 dönüşümü doğruluğu korurken model çıkarımını 4.0–4.4 kat hızlandırmış, uçtan uca hız 79–123 FPS'e ulaşmıştır. Deneyler ayrıca hızlanmanın büyük kısmının yarı hassasiyetten değil, TensorRT'nin grafik optimizasyonlarından geldiğini göstermiştir. İleriki çalışmalarda INT8 kalibrasyonu, Jetson üzerinde ölçüm, küçük nesneler için yüksek çözünürlüklü veya parçalı çıkarım [6] ve personel etiketlerinin iyileştirilmesi planlanmaktadır.

KAYNAKÇA

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.

[2] G. Jocher, J. Qiu, and A. Chaurasia, “Ultralytics YOLO,” 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] Ultralytics, “YOLO26,” Ultralytics Docs, 2025. [Online]. Available: https://docs.ultralytics.com/models/yolo26/

[4] P. Zhu *et al.*, “Detection and tracking meet drones challenge,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7380–7399, 2022.

[5] G.-S. Xia *et al.*, “DOTA: A large-scale dataset for object detection in aerial images,” in *Proc. IEEE CVPR*, 2018, pp. 3974–3983.

[6] F. C. Akyon, S. O. Altinuc, and A. Temizel, “Slicing aided hyper inference and fine-tuning for small object detection,” in *Proc. IEEE ICIP*, 2022, pp. 966–970.

[7] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.

[8] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv:1806.08342*, 2018.

[9] P. Micikevicius *et al.*, “Mixed precision training,” in *Proc. ICLR*, 2018.

[10] NVIDIA, “TensorRT,” [Online]. Available: https://developer.nvidia.com/tensorrt

[11] B. Barz and J. Denzler, “Do we train on test data? Purging CIFAR of near-duplicates,” *J. Imaging*, vol. 6, no. 6, p. 41, 2020.

[12] N. Krawetz, “Kind of like that,” *The Hacker Factor Blog*, 2013. [Online]. Available: https://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html

[13] Roboflow, “Roboflow Universe,” [Online]. Available: https://universe.roboflow.com

[14] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014, pp. 740–755.